
auth-service (:4001)

Request flow, file by file

File-by-file request flow

Event-driven microservices · Go · Kafka · Redis

4 — auth-service (:4001)

Job: users, passwords, JWTs. The only service that reads the users table.

```
index.js
  ▶ await connectDb()          config/db.js
  ▶ createApp().listen()       app.js
  ▶ SIGTERM → close server → disconnectDb()

— POST /auth/signup —————
  ▼
app.js → middleware/requestContext.js
  ▼
routes/auth.routes.js
  |
  |— validateBody(signupSchema)
  |   middleware/validate.js + routes/auth.schemas.js
  |   ▶ role discriminates the shape: an AGENT must send
  |     capabilities { vehicleType, insulated, categories... }
  ▼
controllers/auth.controller.js signup()
  |
  |— bcrypt.hash(password, env.BCRYPT_ROUNDS)    10 rounds
  |— models/User.js      User.create(...)
  |   ▶ unique index on email settles concurrent signups
  |— utils/jwt.js        signToken(user)
  |   ▶ claims: sub, role, email, name, phone
  |— 201 { user, token }   ▶ passwordHash never serialised

— POST /auth/login —————
controllers/auth.controller.js login()
  |— User.findOne({ email })
  |— bcrypt.compare(password, user?.passwordHash ?? DUMMY_HASH)
  |   ▶ the dummy hash burns the same ~10 rounds when the email
  |     does NOT exist, so timing cannot reveal who is registered
  |— 200 { user, token }   | 401 (identical either way)

— GET /auth/me —————
routes/auth.routes.js
  |— requireAuth          middleware/requireAuth.js
  |   └─ utils/jwt.js verifyToken() → req.user
  ▼
controllers/auth.controller.js me()
  └─ User.findById(req.user.id)   ▶ read FRESH, not from the token
```

Who else calls this service: only `agent-location-service`, and only on an agent's first contact, to mirror their capabilities into Redis. Nothing else ever asks "who is this?" — the signed token is the proof.

Appendix — the shared skeleton

Repeated here so this page stands alone. Every Node service has these same files doing the same jobs.

Every Node service has these same files doing the same jobs. Once you know this, each service below is just "the skeleton, plus X".

```
src/
├─ index.js           ▶ bootstrap: connect dependencies, THEN listen
├─ app.js             ▶ build the middleware chain (no port binding)
├─ config/
│   ├─ env.js         ▶ read + validate env vars, fail fast at startup
│   ├─ db.js          ▶ mongoose connection
│   └─ redis.js       ▶ redis client (only where Redis is used)
├─ middleware/
│   ├─ requestContext.js ▶ x-trace-id in/out, attaches req.log
│   ├─ requireAuth.js   ▶ verify JWT → req.user; requireRole(...) gate
│   ├─ validate.js      ▶ schema check → 400 with field-level details
│   └─ errorHandler.js ▶ the ONE place every error is formatted
├─ routes/            *.routes.js ▶ URL → middleware chain → controller
│                      *.schemas.js ▶ the validation schemas
├─ controllers/       ▶ the actual work for one endpoint
├─ models/            ▶ mongoose schemas
└─ utils/
    ├─ jwt.js          ▶ signToken / verifyToken
    ├─ ApiError.js     ▶ typed errors (badRequest, notFound, forbidden...)
    └─ logger.js       ▶ structured JSON logging
```

The universal request pipeline



Why `index.js` and `app.js` are separate. `app.js` builds the Express app but never binds a port; `index.js` connects the database *first*, then listens. That split is what lets the tests drive the app in-process with `supertest` without binding a port or racing on one — and it means a service never accepts a request it cannot serve.

Why the error handler is last. Every failure, from any middleware or controller, funnels into one place. That is also where donation-service cleans up orphaned uploads — putting cleanup on the happy path missed every request that failed validation.